

МИНОБРНАУКИ РОССИИ
Санкт-Петербургский государственный
электротехнический университет
«ЛЭТИ» им. В.И. Ульянова (Ленина)
Кафедра МО ЭВМ

Отчет
по лабораторной работе №2
по дисциплине «Введение в машинное обучение»
Тема: Регрессия.

Студентка гр. 3384

Копасова К. А.

Преподаватель

Жангиров Т. Р.

Санкт-Петербург
2026

Цель работы: Изучить линейную и нелинейную регрессии.

Задание: Вариант 1В - lab2_lin1.csv и lab2_poly2.csv.

1. Линейная регрессия

1.1. Загрузите набор данных соответствующей цифре вашего варианта. Убедитесь, что загрузка прошла корректно.

1.2. Используя `train_test_split` разбейте выборку на обучающую и тестовую. Проверьте, что тестовая выборка соответствует обучающей. Можно использовать диаграммы рассеяния/нормированные гистограммы/`boxplot`/`violin plot`.

1.3. Проведите линейную регрессию используя `LinearRegression`. Получите коэффициенты регрессии и объясните полученные результаты.

1.4. Для обучающей и тестовой выборки рассчитайте коэффициент детерминации, `MARE`, `MAE`. Объясните полученные значений метрик. Сравните метрики для обучающей и тестовой выборки, сделайте выводы о качестве обобщения полученной модели.

1.5. Повторите пункты 1.3 и 1.4 для модификаций линейной регрессии:

1.5.1. Лассо регрессия. Самостоятельно подберите гиперпараметры

1.5.2. Гребневая регрессия. Самостоятельно подберите гиперпараметры

1.5.3. `ElasticNet`. Самостоятельно подберите гиперпараметры

1.6. Постройте сводную таблицу для рассчитываемых метрик, и используемых методов (с разделением на обучающую и тестовую выборку). По таблице сделайте выводы о том, какой вид регрессии дал лучшую модель. Опишите какие проблемы могут возникнуть при применении каждой модели. Для объяснения результатов можно построить “карту высот (`heat map`)” с отображением корреляции признаков.

1.7. Для модели, которая дала лучшие результаты, постройте диаграмму рассеяния между предикторами и откликом. На диаграмме изобразите какое значение должно быть, и какое предсказывается. Визуально оцените качество построенного регрессора.

2. Нелинейная регрессия.

2.1. Загрузите набор данных соответствующей букве вашего варианта. Убедитесь, что загрузка прошла корректно.

2.2. Используя `train_test_split` разбейте выборку на обучающую и тестовую. Проверьте, что тестовая выборка соответствует обучающей.

2.3. Проверьте работу стандартной линейной регрессии на загруженных данных. Постройте диаграмму рассеяния данных с выделенной полученной линией регрессии. Объясните полученный результат.

2.4. Конструируя полиномиальные признаки для разных степеней полинома найдите степень полинома наилучшим образом аппроксимирующая данные. Постройте график зависимости коэффициента детерминации от степени полинома(на одном графике изобразите линии для обучающей и тестовой выборки отдельно). Сделайте вывод о том, при какой степени полинома модель начинает переобучаться.

2.5. Для выбранной степени полинома, рассчитайте и проанализируйте полученные коэффициенты. Рассчитай значение метрик коэффициент детерминации, MAPE, MAE.

2.6. Для выбранной степени полинома, постройте диаграмму рассеяния данных с линией соответствующей полученному полиному. Сделайте выводы о качестве аппроксимации.

2.7. Для выбранной степени полинома, решите задачу нелинейной регрессии без конструирования полиномиальных признаков и используя библиотеку TensorFlow.

2.8. Рассчитай метрики коэффициент детерминации, MAPE, MAE, а также постройте диаграмму рассеяния для данных линией соответствующей полученному полиному, для модели полученной при помощи TensorFlow.

2.9. Сравните результаты полученные с/без конструирования полиномиальных признаков.

Выполнение работы

1. Линейная регрессия

1.1. Загрузка набора данных и проверка его корректности.

Прочитаем файл lab2_lin1.csv через метод read_csv() библиотеки pandas.

```
import pandas as pd
data = pd.read_csv('lab2_lin1.csv')
print(data.head())
```

Результат работы показан на рис. 1.1.

	Unnamed: 0	WorkingAges	LateTimes	KPI	Salary
0	0	20.0	21.0	4.37	3619.6
1	1	24.0	14.0	5.49	5333.7
2	2	22.0	14.0	3.53	3022.7
3	3	26.0	13.0	4.90	5079.7
4	4	18.0	20.0	5.75	4656.0

Рисунок 1.1 - Демонстрация набора данных.

Набор данных содержит следующие столбцы:

- 1) Unnamed: 0 - индекс строки;
- 2) WorkingAges - стаж работника;
- 3) LateTimes - количество опозданий сотрудника;
- 4) KPI - показатель эффективности сотрудника;
- 5) Salary - заработная плата сотрудника.

Признаки WorkingAges, LateTimes и KPI потенциально могут влиять на заработную плату сотрудника. Пусть Salary - целевая переменная и остальные признаки будут использоваться для ее предсказания.

1.2. Разбиение выборки на обучающую и тестовую. Проверка соответствия выборок.

Разделим данные на независимые признаки (X, будет содержать столбцы WorkingAges, LateTimes и KPI) и на целевую переменную (y, будет содержать столбец Salary). После разобьем данные на обучающую и тестовую выборки с помощью функции train_test_split() из библиотеки sklearn. Размер тестовой выборки составляет 20% от всего набора данных.

```
from sklearn.model_selection import train_test_split

X = data[['WorkingAges', 'LateTimes', 'KPI']]
```

```

y = data['Salary']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=41)
print(X.shape, X_train.shape, X_test.shape)

```

Результат работы показан на рис. 1.2.1.

```
(500, 3) (400, 3) (100, 3)
```

Рисунок 1.2.1 - Размеры изначальной, обучающей и тестовой выборки соответственно.

В результате разбиения получены следующие размеры выборок: (500, 3) - исходный набор данных, содержит 500 объектов по 3 признака; (400, 3) - обучающая выборка; (100, 3) - тестовая выборка. Таким образом, 80% данных используется для обучения, а остальные 20% - для проверки качества обучения.

Проверим распределение данных с помощью нормированной гистограммы.

```

import matplotlib.pyplot as plt

plt.figure(figsize=(15, 4))

for i, col in enumerate(X.columns):
    plt.subplot(1, 3, i+1)
    plt.hist(X_train[col], bins=20, alpha=0.6, label='Train', density=True)
    plt.hist(X_test[col], bins=20, alpha=0.6, label='Test', density=True)
    plt.title(col)
    plt.legend()

plt.tight_layout()

```

Результат работы показан на рис. 1.2.2.

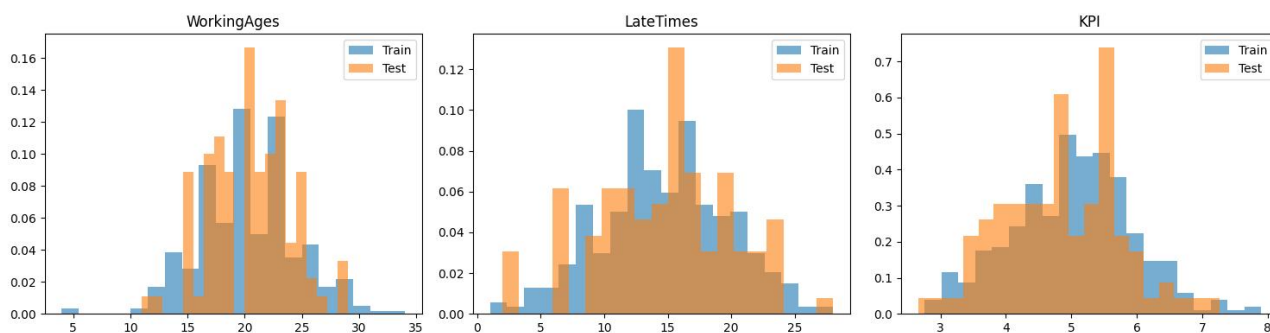


Рисунок 1.2.2 - Сравнение распределений признаков в обучающей и тестовой выборках.

Из всех графиков видно, что распределения всех признаков в обучающей и тестовой выборке имеют схожий характер.

1.3. Проведение линейной регрессии.

Построим модель линейной регрессии с использованием класса `LinearRegression` из библиотеки `sklearn`.

```
from sklearn.linear_model import LinearRegression

model_lin_reg = LinearRegression()
model_lin_reg.fit(X_train, y_train)
print(model_lin_reg.coef_)
print(model_lin_reg.intercept_)
```

Результат работы показан на рис. 1.3.

```
[[1.72494491e+02 1.51876582e-01 1.06673991e+03]]
[-4583.2116057]
```

Рисунок 1.3 - Коэффициенты и свободный член для линейной регрессии.

Модель линейной регрессии в задаче имеет вид (1):

$$Salary = b_0 + b_1 WorkingAges + b_2 LateTimes + b_3 KPI \quad (1)$$

где b_0 - свободный член;

b_1, b_2, b_3 - коэффициенты регрессии модели.

Полученные коэффициенты показывают степень влияния признаков на целевую переменную. Сделаем следующие выводы:

1) $b_1 = 172.494491$ показывает, что с увеличением стажа на 1 год зарплата растет в соответствии с этим коэффициентом;

2) $b_2 = 0.151877$ показывает, что опоздания практически не влияют на зарплату;

3) $b_3 = 1066.73991$ показывает самое сильное влияние эффективности на зарплату;

4) $b_0 = -4583.211606$ показывает зарплату при нулевых значениях всех признаков (поскольку модель не ограничена - это просто математический результат вычислений).

1.4. Расчет коэффициента детерминации, MAPE и MAE.

Коэффициент детерминации показывает, какая доля вариации целевой переменной объясняется моделью (1 - модель объясняет почти все изменения, 0 - модель почти ничего не объясняет). MAE (средняя абсолютная ошибка)

показывает среднюю величину ошибки предсказания в тех же единицах, что и целевая переменная. MAPE (средняя абсолютная процентная ошибка) показывает среднюю ошибку в процентах.

```
from sklearn.metrics import r2_score, mean_absolute_error
import numpy as np

y_train_pred = model_lin_reg.predict(X_train)
y_test_pred = model_lin_reg.predict(X_test)

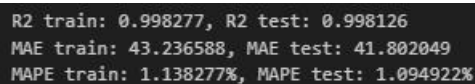
r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)

mae_train = mean_absolute_error(y_train, y_train_pred)
mae_test = mean_absolute_error(y_test, y_test_pred)

mape_train = np.mean(np.abs((y_train - y_train_pred) / y_train)) * 100
mape_test = np.mean(np.abs((y_test - y_test_pred) / y_test)) * 100

print(f"R2 train: {r2_train:.3f}, R2 test: {r2_test:.3f}")
print(f"MAE train: {mae_train:.2f}, MAE test: {mae_test:.2f}")
print(f"MAPE train: {mape_train:.2f}%, MAPE test: {mape_test:.2f}%")
```

Результат работы показан на рис. 1.4.



```
R2 train: 0.998277, R2 test: 0.998126
MAE train: 43.236588, MAE test: 41.802049
MAPE train: 1.138277%, MAPE test: 1.094922%
```

Рисунок 1.4 - R2, MAE и MAPE для линейной регрессии.

Модель объясняет 99.8% вариации зарплаты на обучающей выборке и 99.8% на тестовой - модель хорошо улавливает зависимости и правильно обобщает. Средняя ошибка предсказания около 43 ед. зарплаты на обучающей выборке и 41 - на тестовой. MAPE на обучающей и тестовой выборках ~1%, что говорит о высокой точности модели.

1.5. Повторение пунктов 1.3 и 1.4 для модификаций линейной регрессии:

1.5.1. Лассо регрессия.

Лассо регрессия - это метод линейной регрессии с регуляризацией, который сохраняет наиболее значимые признаки для предсказания целевой модели.

Построим модель лассо регрессии с использованием класса Lasso из библиотеки sklearn и подберем для нее гиперпараметр α .

```
from sklearn.linear_model import Lasso
from sklearn.model_selection import GridSearchCV

alphas = [0.001, 0.01, 0.1, 1, 10]
```

```
lasso = Lasso(max_iter=10000)
grid_lasso = GridSearchCV(lasso, param_grid={'alpha': alphas}, cv=5)
grid_lasso.fit(X_train, y_train)

model_lasso = grid_lasso.best_estimator_

print(model_lasso.alpha)
print(model_lasso.coef_)
print(model_lasso.intercept_)
```

Результат работы показан на рис. 1.5.1.1.

```
0.001
[1.72494411e+02 1.51836544e-01 1.06673873e+03]
-4583.203471233154
```

Рисунок 1.5.1.1 - α , коэффициенты и свободный член для регрессии лассо.

$\alpha = 0.001$, $b_1 = 172.494411$, $b_2 = 0.151837$, $b_3 = 1066.73873$, $b_0 = -4583.203471$.

Полученные коэффициенты практически аналогичны с коэффициентами обычной линейной регрессией, что говорит о том, что регуляризация по умолчанию практически не изменила коэффициенты модели, т. к. все они значимы для предсказания зарплаты.

Рассчитаем коэффициент детерминации, MAE и MAPE для лассо регрессии.

```
y_train_pred = model_lasso.predict(X_train)
y_test_pred = model_lasso.predict(X_test)

r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)

mae_train = mean_absolute_error(y_train, y_train_pred)
mae_test = mean_absolute_error(y_test, y_test_pred)

mape_train = np.mean(np.abs((y_train - y_train_pred) / y_train)) * 100
mape_test = np.mean(np.abs((y_test - y_test_pred) / y_test)) * 100

print(f"R2 train: {r2_train:.3f}, R2 test: {r2_test:.3f}")
print(f"MAE train: {mae_train:.2f}, MAE test: {mae_test:.2f}")
print(f"MAPE train: {mape_train:.2f}%, MAPE test: {mape_test:.2f}%")
```

Результат работы показан на рис. 1.5.1.2.

```
R2 train: 0.998277, R2 test: 0.998126
MAE train: 43.236595, MAE test: 41.802163
MAPE train: 1.138274%, MAPE test: 1.094926%
```

Рисунок 1.5.1.2 - R2, MAE и MAPE для лассо регрессии.

Результат практически совпадает с линейной регрессией.

Модель Lasso на данной выборке аналогична линейной регрессии, поскольку все признаки оказались значимыми и регуляризация не исключила ни одного из них.

1.5.2. Гребневая регрессия.

Гребневая регрессия - это метод линейной регрессии с регуляризацией, который снижает влияние малоинформативных признаков и предотвращает переобучение, сохраняя все признаки в модели.

Построим модель гребневой регрессии с использованием класса Ridge из библиотеки sklearn и подберем для нее гиперпараметр α .

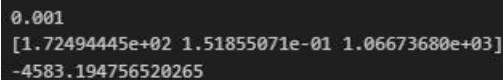
```
from sklearn.linear_model import Ridge

ridge = Ridge(max_iter=10000)
grid_ridge = GridSearchCV(ridge, param_grid={'alpha': alphas}, cv=5)
grid_ridge.fit(X_train, y_train)

model_ridge = grid_ridge.best_estimator_

print(model_ridge.alpha)
print(model_ridge.coef_)
print(model_ridge.intercept_)
```

Результат работы показан на рис. 1.5.2.1.



```
0.001
[1.72494445e+02 1.51855071e-01 1.06673680e+03]
-4583.194756520265
```

Рисунок 1.5.2.1 - α , коэффициенты и свободный член для гребневой регрессии.

$$\alpha = 0.001, b_1 = 172.494445, b_2 = 0.151855, b_3 = 1066.73680, b_0 = -4583.194757.$$

Полученные коэффициенты практически совпадают с коэффициентами линейной и лассо регрессии, что говорит о том, что все признаки влияют на предсказания зарплаты, и поэтому регуляризация Ridge немного уменьшила величины коэффициентов.

Рассчитаем коэффициент детерминации, MAE и MAPE для гребневой регрессии.

```
y_train_pred = model_ridge.predict(X_train)
y_test_pred = model_ridge.predict(X_test)

r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)
```

```

mae_train = mean_absolute_error(y_train, y_train_pred)
mae_test = mean_absolute_error(y_test, y_test_pred)

mape_train = np.mean(np.abs((y_train - y_train_pred) / y_train)) * 100
mape_test = np.mean(np.abs((y_test - y_test_pred) / y_test)) * 100

print(f"R2 train: {r2_train:.6f}, R2 test: {r2_test:.6f}")
print(f"MAE train: {mae_train:.6f}, MAE test: {mae_test:.6f}")
print(f"MAPE train: {mape_train:.6f}%, MAPE test: {mape_test:.6f}%")

```

Результат работы показан на рис. 1.5.2.2.

```

R2 train: 0.998277, R2 test: 0.998126
MAE train: 43.236623, MAE test: 41.802382
MAPE train: 1.138272%, MAPE test: 1.094932%

```

Рисунок 1.5.2.2 - R2, MAE и MAPE для гребневой регрессии.

Результат практически совпадает с линейной и лассо регрессиями.

Модель Ridge на данной выборке ведёт себя аналогично линейной регрессии, поскольку все признаки значимы, а регуляризация лишь слегка снижает величину коэффициентов, не исключая ни одного признака.

1.5.3. ElasticNet.

ElasticNet - это метод линейной регрессии, который объединяет свойства лассо и гребневой регуляризаций (L1 и L2 регуляризация соответственно). Он одновременно снижает влияние малоинформативных признаков и сохраняет наиболее значимые признаки для предсказания целевой переменной.

Построим модель ElasticNet с использованием класса ElasticNet из библиотеки sklearn и подберем для нее гиперпараметры (α , l1_ratio - баланс регуляризаций).

```

from sklearn.linear_model import ElasticNet

l1_ratios = [0.1, 0.3, 0.5, 0.7, 0.9]

param_grid = {
    'alpha': alphas,
    'l1_ratio': l1_ratios
}

elastic = ElasticNet(max_iter=10000)
grid_elastic = GridSearchCV(elastic, param_grid=param_grid, cv=5)
grid_elastic.fit(X_train, y_train)

model_elastic = grid_elastic.best_estimator_

print(model_elastic.alpha)
print(model_elastic.l1_ratio)
print(model_elastic.coef_)
print(model_elastic.intercept_)

```

Результат работы показан на рис. 1.5.3.1.

```
0.001
0.9
[1.72492590e+02 1.50980899e-01 1.06661452e+03]
-4582.53036458091
```

Рисунок 1.5.3.1 - гиперпараметры, коэффициенты и свободный член для ElasticNet.

$\alpha = 0.001$, $l1_ratio = 0.9$, $b_1 = 172.492590$, $b_2 = 0.150981$, $b_3 = 1066.614520$, $b_0 = -4582.530365$.

Рассчитаем коэффициент детерминации, MAE и MAPE для ElasticNet .

```
y_train_pred = model_elastic.predict(X_train)
y_test_pred = model_elastic.predict(X_test)

r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)

mae_train = mean_absolute_error(y_train, y_train_pred)
mae_test = mean_absolute_error(y_test, y_test_pred)

mape_train = np.mean(np.abs((y_train - y_train_pred) / y_train)) * 100
mape_test = np.mean(np.abs((y_test - y_test_pred) / y_test)) * 100

print(f"R2 train: {r2_train:.6f}, R2 test: {r2_test:.6f}")
print(f"MAE train: {mae_train:.6f}, MAE test: {mae_test:.6f}")
print(f"MAPE train: {mape_train:.6f}%, MAPE test: {mape_test:.6f}%")
```

Результат работы показан на рис. 1.5.3.2.

```
R2 train: 0.998277, R2 test: 0.998125
MAE train: 43.238005, MAE test: 41.815448
MAPE train: 1.138075%, MAPE test: 1.095335%
```

Рисунок 1.5.3.2 - R2, MAE и MAPE для ElasticNet.

Полученные коэффициенты показывают, что модель ElasticNet сочетает свойства Lasso и Ridge. Регуляризация уменьшает величины коэффициентов и может снижать влияние некоторых признаков, однако в данном наборе данных влияние регуляризации незначительно, поэтому коэффициенты остаются близкими к коэффициентам обычной линейной регрессии. По значениям метрик качества модель показывает результаты, практически совпадающие с линейной, Lasso и Ridge регрессиями. Небольшое снижение показателей

объясняется более сильным сжатием коэффициентов из-за комбинированной регуляризации.

1.6. Построение таблицы для рассчитываемых метрик и используемых методов.

Построим таблицу по всем методам регрессии и вычисленным признакам (см. Таб. 1).

Таблица 1 - Сравнение метрик качества для различных методов регрессии.

Метод регрессии	R2 train	R2 test	MAE train	MAE test	MAPE train, %	MAPE test, %
Линейная	0.998277	0.998126	43.236588	41.802049	1.138277	1.094922
Лассо	0.998277	0.998126	43.236595	41.802163	1.138274	1.094926
Гребневая	0.998277	0.998126	43.236623	41.802382	1.138272	1.094932
ElasticNet	0.998277	0.998125	43.238005	41.815448	1.138075	1.095335

По данным таблицы 1 видно, что все модели показывают практически одинаковые значения коэффициента детерминации R2, а также метрик MAE и MAPE. Наилучшие результаты показывает линейная регрессия, но различия между моделями минимальны. Таким образом, зависимость между признаками и целевой переменной в данных близка к линейной и хорошо описывается простой линейной моделью.

Модели Lasso и Ridge дают практически идентичные результаты по сравнению с линейной регрессией. Это объясняется тем, что все признаки оказываются значимыми для предсказания зарплаты, поэтому регуляризация практически не изменяет коэффициенты модели.

Модель ElasticNet показывает немного хуже значения метрик, что связано с более сильным влиянием регуляризации, которая дополнительно уменьшает коэффициенты признаков.

При применении различных моделей могут возникать следующие проблемы:

1) линейная регрессия чувствительна к мультиколлинеарности признаков;

- 2) Lasso может занулять коэффициенты и исключать признаки из модели, что может привести к потере части информации;
- 3) Ridge может сохранять все признаки, но уменьшать их влияние;
- 4) ElasticNet требует более тщательного подбора гиперпараметров и может ухудшать качество модели при неправильной настройке.

Построим heatmap для проверки корреляции между признаками.

```
import seaborn as sns

corr = data[['WorkingAges', 'LateTimes', 'KPI', 'Salary']].corr()

plt.figure(figsize=(6,5))
sns.heatmap(corr, annot=True, cmap='coolwarm', fmt=".2f")

plt.title("heatmap")
plt.show()
```

Результат работы показан на рис. 1.6.

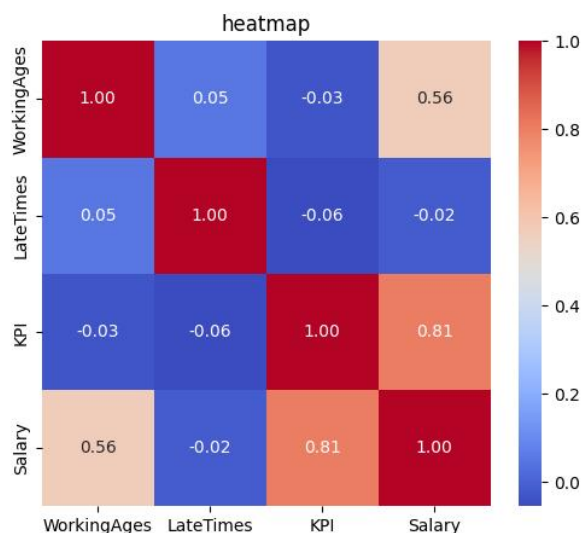


Рисунок 1.6 - Корреляция между всеми признаками набора данных.

По тепловой карте видно, что показатель KPI имеет наиболее сильную положительную корреляцию с заработной платой (Salary). Признак WorkingAges также положительно коррелирует с зарплатой, но слабее. Признак LateTimes имеет слабую связь с целевой переменной, что подтверждает ранее полученные коэффициенты регрессии.

1.7. Построение диаграммы рассеяния между предикторами и откликом для модели с лучшим результатом.

В п. 1.6. линейная регрессия была определена лучшей моделью, поэтому построим диаграмму рассеяния для нее.

```
import matplotlib.pyplot as plt

y_pred = model_lin_reg.predict(X_test)

plt.figure(figsize=(6,6))

plt.scatter(y_test, y_pred)

plt.plot([y_test.min(), y_test.max()],
         [y_test.min(), y_test.max()],
         linestyle='--')

plt.xlabel("Реальная зарплата (Salary)")
plt.ylabel("Предсказанная зарплата")
plt.title("Диаграмма рассеяния реальных и предсказанных значений")
plt.show()
```

Результат работы показан на рис. 1.7.

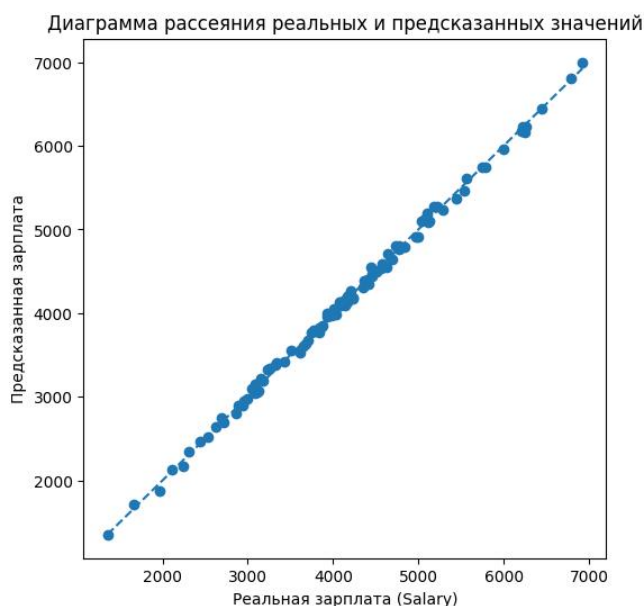


Рисунок 1.7 - Диаграмма рассеяния для линейной регрессии.

По диаграмме видно, что точки расположены близко к диагональной линии, что говорит о высокой точности модели. Предсказанные значения практически совпадают с реальными значениями заработной платы. Это подтверждает ранее полученные значения метрик качества.

Построенная модель линейной регрессии хорошо описывает зависимость между признаками и целевой переменной и обладает хорошей обобщающей способностью.

2. Нелинейная регрессия

2.1. Загрузка набора данных и проверка его корректности.

Прочитаем файл lab2_poly2.csv через метод read_csv() библиотеки pandas.

```
data_poly = pd.read_csv('lab2_poly2.csv')
print(data_poly.head())
```

Результат работы показан на рис. 2.1.

	Unnamed: 0	Experience	Salary
0	0	34.148659	35186.143515
1	1	39.429129	45366.424097
2	2	13.837876	65664.538074
3	3	31.028990	36133.821589
4	4	25.258173	53814.594836

Рисунок 2.1 - Демонстрация набора данных.

Набор данных содержит следующие столбцы:

- 1) Unnamed: 0 - индекс строки;
- 2) Experience - опыт работы сотрудника;
- 3) Salary - заработная плата сотрудника.

Признак Experience используется в качестве независимой переменной, а Salary - в качестве целевой переменной, которую требуется предсказать.

2.2. Разбиение выборки на обучающую и тестовую. Проверка соответствия выборок.

Разделим данные на независимые признаки (X, будет содержать столбец Experience) и на целевую переменную (y, будет содержать столбец Salary). После разобьем данные на обучающую и тестовую выборки с помощью функции train_test_split() из библиотеки sklearn. Размер тестовой выборки составляет 20% от всего набора данных.

```
X = data_poly[['Experience']]
y = data_poly['Salary']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
print(X.shape, X_train.shape, X_test.shape)
```

Результат работы показан на рис. 2.2.1.

```
(500, 1) (400, 1) (100, 1)
```

Рисунок 2.2.1 - Размеры изначальной, обучающей и тестовой выборки соответственно.

В результате разбиения получены аналогичные размеры выборок, что и в п. 1.2 (за исключением того, что признака не 3, а только 1) - 80% данных используется для обучения, а остальные 20% - для проверки качества обучения.

Проверим распределение данных с помощью нормированной гистограммы.

```
plt.figure(figsize=(8, 5))
plt.hist(X_train['Experience'], bins=20, alpha=0.6, label='Train', density=True)
plt.hist(X_test['Experience'], bins=20, alpha=0.6, label='Test', density=True)
plt.title('Experience')
plt.legend()
plt.tight_layout()
```

Результат работы показан на рис. 2.2.2.

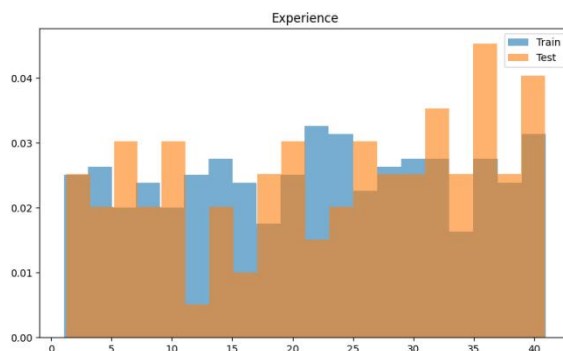


Рисунок 2.2.2 - Сравнение распределений признаков в обучающей и тестовой выборках.

Гистограммы показывают, что обучающая и тестовая выборки имеют схожие распределения: одинаковый диапазон значений признаков, аналогичную структуру с локальными экстремумами. Данные обеих выборок равномерно перекрываются - это показывает корректность разбиения.

2.3. Проверка загруженных данных на линейной регрессии. Построение диаграммы рассеяния данных.

Построим диаграмму рассеяния линейной регрессии на нелинейных данных.

```
model_lin_reg_poly = LinearRegression()
model_lin_reg_poly.fit(X_train, y_train)
y_pred_poly = model_lin_reg_poly.predict(X_train)

plt.figure(figsize=(6, 5))
```



```
plt.scatter(X, y)
plt.plot(X_train, y_pred_poly)
plt.xlabel("Experience")
plt.ylabel("Salary")
plt.title("Линейная регрессия")
plt.show()
```

Результат работы показан на рис. 2.3.

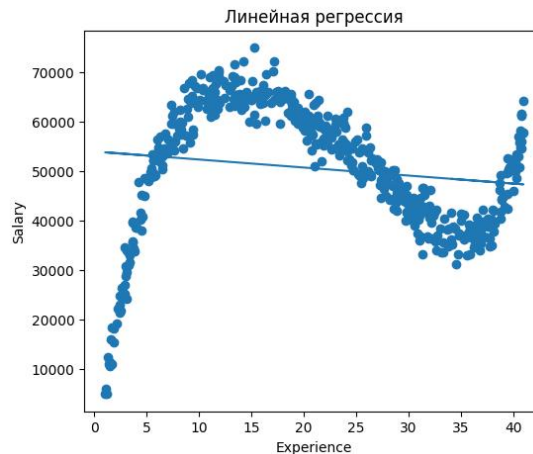


Рисунок 2.3 - Диаграмма рассеяния данных и линия линейной регрессии.

По диаграмме видно, что зависимость между переменными Experience и Salary имеет нелинейный характер. Линия линейной регрессии не отражает форму распределения данных и проходит через точки с заметными отклонениями.

Линейная модель недостаточно хорошо описывает зависимость между опытом работы и заработной платой. Для более точного описания данных необходимо использовать нелинейную регрессию.

2.4. Нахождение степени полинома.

Создадим полиномиальные признаки, чтобы линейная регрессия могла описывать нелинейную зависимость между переменными. Обучим модели для разных степеней полинома и сравним коэффициент детерминации на обучающей и тестовой выборках, чтобы определить оптимальную сложность модели и момент начала переобучения.

```
from sklearn.preprocessing import PolynomialFeatures

r2_train_list = []
r2_test_list = []

for d in range(1, 20):
```

```

poly = PolynomialFeatures(degree=d)

X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)

y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)

r2_train_list.append(r2_score(y_train, y_train_pred))
r2_test_list.append(r2_score(y_test, y_test_pred))

plt.figure(figsize=(7,5))
plt.plot(range(1, 20), r2_train_list, marker='o', label="Train")
plt.plot(range(1, 20), r2_test_list, marker='o', label="Test")
plt.xticks(range(2, 20))
plt.yticks(np.arange(0.0, 1.01, 0.1))
plt.xlabel("Степень полинома")
plt.ylabel("R2")
plt.title("Зависимость коэффициента детерминации от степени полинома")
plt.legend()
plt.show()

```

Результат работы показан на рис. 2.4.

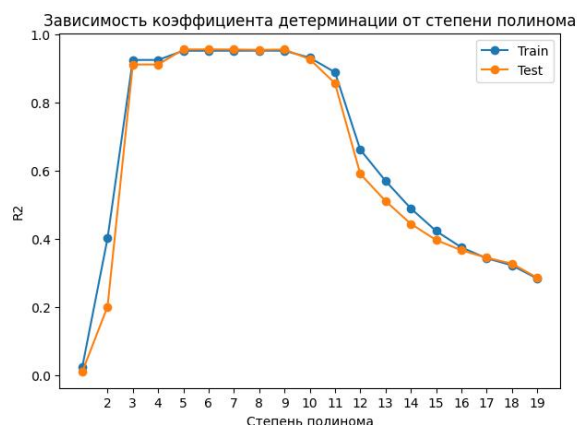


Рисунок 2.4 - Зависимость коэффициента детерминации от степени полинома.

До 5 степени полинома значение R^2 на обучающей и тестовой выборках быстро растет (с 0.0 до > 0.9) и практически совпадает, что говорит о недостаточной сложности модели. В диапазоне 5-9 степеней значение R^2 стабилизируется на высоком уровне (~ 0.95) для обеих выборок, что указывает на оптимальную степень полинома. Начиная с 10 степени, значение R^2 на тестовой выборке начинает резко уменьшаться, а разрыв между обучающей и тестовой выборками растет - это говорит о переобучении модели.

Выберем степень полинома 6, так как в этой точке коэффициент детерминации на обучающей и тестовой выборках принимает высокие и близкие значения.

2.5. Расчет и анализ полученных коэффициентов. Расчет значений метрик коэффициента детерминации, MAPE и MAE.

Создадим полиномиальные признаки (6) и обучим линейную регрессию на них.

```
degree = 6

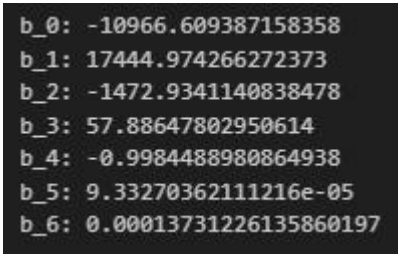
poly = PolynomialFeatures(degree=degree)
X_train_poly = poly.fit_transform(X_train)
X_test_poly = poly.transform(X_test)

model = LinearRegression()
model.fit(X_train_poly, y_train)

y_train_pred = model.predict(X_train_poly)
y_test_pred = model.predict(X_test_poly)

print(f"b_0: {model.intercept_}")
for i, coef in enumerate(model.coef_):
    if i!=0:
        print(f"b_{i}: {coef}")
```

Результат работы показан на рис. 2.5.1.



```
b_0: -10966.609387158358
b_1: 17444.974266272373
b_2: -1472.9341140838478
b_3: 57.88647802950614
b_4: -0.9984488980864938
b_5: 9.33270362111216e-05
b_6: 0.00013731226135860197
```

Рисунок 2.5.1 - Полученные коэффициенты.

Проанализируем коэффициенты:

- 1) $b_0 = -10966.609$ - базовое смещение модели;
- 2) $b_1 = 17444.974$ показывает, что основной рост зависимой переменной вносит линейный компонент, это основной признак модели;
- 3) $b_2 = -1472.934$ показывает, что квадратичный компонент уменьшает предсказание при росте признака - небольшая кривизна модели, которая корректирует линейный рост;

4) $b_3 = 57.886$ показывает минимальное влияние кубического компонента, немного корректирует кривизну;

5) $b_4 = -0.998$ показывает практически нулевое влияние четвертой степени;

6) $b_5 = 9.33 \cdot 10^{-5}$ и $b_6 = 1.37 \cdot 10^{-4}$ показывают, что пятая и шестая степени практически не влияют на предсказание. Это подтверждает, что степень 6 подходит - увеличение степени могла бы усложнить модель.

Также видим, что знаки коэффициентов чередуются, что может указывать на полиномиальную аппроксимацию.

Рассчитаем метрики коэффициента детерминации, MAE и MAPE для полученной модели.

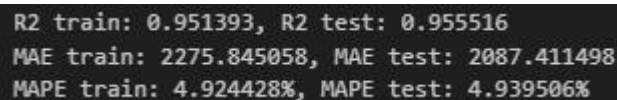
```
r2_train = r2_score(y_train, y_train_pred)
r2_test = r2_score(y_test, y_test_pred)

mae_train = mean_absolute_error(y_train, y_train_pred)
mae_test = mean_absolute_error(y_test, y_test_pred)

mape_train = np.mean(np.abs((y_train - y_train_pred) / y_train)) * 100
mape_test = np.mean(np.abs((y_test - y_test_pred) / y_test)) * 100

print(f"R2 train: {r2_train:.6f}, R2 test: {r2_test:.6f}")
print(f"MAE train: {mae_train:.6f}, MAE test: {mae_test:.6f}")
print(f"MAPE train: {mape_train:.6f}%, MAPE test: {mape_test:.6f}%")
```

Результат работы показан на рис. 2.5.2.



```
R2 train: 0.951393, R2 test: 0.955516
MAE train: 2275.845058, MAE test: 2087.411498
MAPE train: 4.924428%, MAPE test: 4.939506%
```

Рисунок 2.5.2 - Метрики R2, MAE и MAPE для полиномиальной регрессии.

Модель объясняет 95.1% данных на обучающей выборке и 95.6% на тестовой - модель не переобучена, так как метрики на тестовой выборке не хуже, чем на обучающей. Небольшое улучшение качества на тесте может быть связано с особенностями разбиения данных. Средняя ошибка предсказания 2275.85 на обучающей выборке и 2087.41 - на тестовой. MAPE на обучающей и тестовой выборках ~5%, что указывает на высокую точность прогноза модели.

2.6. Построение диаграммы рассеяния данных с линией полинома.

Построим график с точками рассеяния данных и с предсказанием обученной модели.

```
plt.scatter(X_train, y_train, alpha=0.6, label='Исходные данные', s=30)

X_train_sorted = np.sort(X_train, axis=0)
X_train_poly_sorted = poly.transform(X_train_sorted)
y_pred_sorted = model.predict(X_train_poly_sorted)

plt.plot(X_train_sorted, y_pred_sorted, color='red', linewidth=2,
label='Полином 6 степени')

plt.xlabel('Признак X')
plt.ylabel('Целевая переменная y')
plt.title('Диаграмма рассеяния с линией полиномиальной регрессии')
plt.legend()
plt.grid(alpha=0.3)
plt.show()
```

Результат работы показан на рис. 2.6.

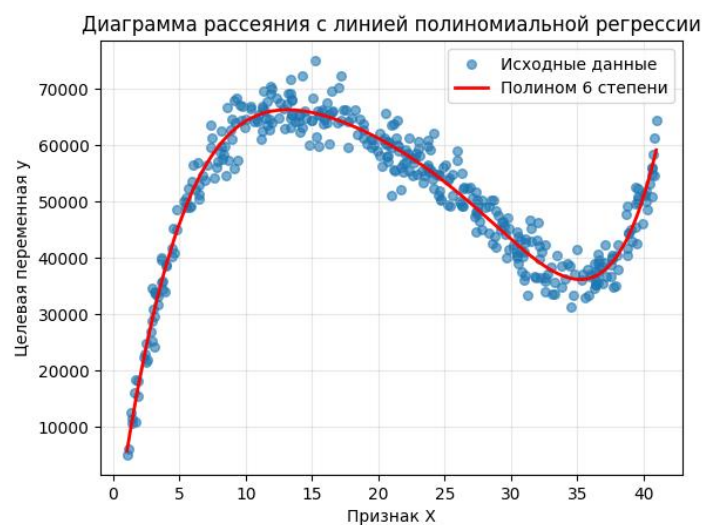


Рисунок 2.6 - Диаграмма рассеяния с линией полиномиальной регрессии.

Полином 6 степени хорошо описывает данные: кривая полинома проходит через центр облака точек и правильно повторяет нелинейный характер данных. Точки равномерно распределены относительно кривой по всей длине графика - нет областей, где модель занижает или завышает предсказания. Разброс точек выглядит случайным, без явных паттернов. Плавность кривой без резких колебаний подтверждает отсутствие переобучения. Визуальное качество моделирования соответствует высоким значениям метрик ($R^2 \sim 95.5\%$, $MAPE \sim 5\%$), рассчитанных в п. 2.5.

2.7. Решение задачи нелинейной регрессии через библиотеку TensorFlow.

Решим задачу нелинейной регрессии через библиотеку TensorFlow.

```
import tensorflow as tf
```

```

from sklearn.preprocessing import StandardScaler

scaler_X = StandardScaler()
scaler_y = StandardScaler()

X_train_scaled = scaler_X.fit_transform(X_train)
X_test_scaled = scaler_X.transform(X_test)

y_train_scaled = scaler_y.fit_transform(y_train.values.reshape(-1,1))
y_test_scaled = scaler_y.transform(y_test.values.reshape(-1,1))

X_train_tf = tf.constant(X_train_scaled, dtype=tf.float32)
y_train_tf = tf.constant(y_train_scaled, dtype=tf.float32)

num_coef = 7
w = tf.Variable(tf.zeros([num_coef], dtype=tf.float32))

def model_tf(x):
    y_pred = 0
    for i in range(num_coef):
        y_pred += w[i] * x**i
    return y_pred

optimizer = tf.optimizers.Adam(0.01)

for epoch in range(5000):
    with tf.GradientTape() as tape:
        y_pred = model_tf(X_train_scaled)
        loss = tf.reduce_mean((y_train_scaled - y_pred)**2)

    grads = tape.gradient(loss, [w])
    optimizer.apply_gradients(zip(grads, [w]))

w_values = w.numpy()
for i in range(len(w_values)):
    print(f"b_{i}: {w_values[i]}")

```

Результат работы показан на рис. 2.7.

```

b_0: 0.6590833067893982
b_1: -1.264103651046753
b_2: -0.6078616976737976
b_3: -0.02942206710577011
b_4: -0.05630442872643471
b_5: 0.28190600872039795
b_6: 0.02516642026603222

```

Рисунок 2.7 - Полученные коэффициенты.

Сначала данные X и y стандартизируются с помощью `StandardScaler` для приведения признаков к единому масштабу, что важно для работы градиентного спуска. Затем данные преобразуются в тензоры TensorFlow с типом `float32`. Инициализируются веса модели w (7 коэффициентов для полинома 6-й степени) нулевыми значениями с возможностью обучения.

Функция `model_tf` вычисляет полиномиальную зависимость как сумму произведений весов на степени признака. Оптимизатор Adam настраивается со скоростью обучения 0.01 для обновления весов. В цикле обучения (5000 эпох) с использованием GradientTape автоматически вычисляются градиенты функции потерь MSE по весам, после чего оптимизатор применяет эти градиенты для минимизации ошибки. После завершения обучения полученные коэффициенты извлекаются из тензора и выводятся.

Проанализируем коэффициенты:

- 1) $b_0 = 0.659$ - базовое смещение модели;
- 2) $b_1 = -1.264$ - линейный компонент, по модулю самый крупный коэффициент, отражает основной тренд изменения;
- 3) $b_2 = -0.608$ - квадратичный член, создает небольшую кривизну, добавляя изогнутую форму зависимости;
- 4) $b_3 = -0.029$ - кубический компонент, минимально корректирует кривизну модели;
- 5) $b_4 = -0.056$ - четвертая степень, практически не влияет на регрессию;
- 6) $b_5 = 0.282$ и $b_6 = 0.025$ показывают, что пятая степень добавляет небольшую выпуклость для больших значений, а шестая степень немного корректирует крайние значения.

Рассчитаем метрики коэффициента детерминации, MAE и MAPE для полученной модели.

2.8. Расчет метрик коэффициента детерминации, MAPE, MAE.

Построение диаграммы рассеяния для полученных данных.

Рассчитаем метрики у TensorFlow модели из п. 2.7.

```
y_train_pred_scaled = model_tf(X_train_tf).numpy()
y_test_pred_scaled = model_tf(tf.constant(X_test_scaled,
dtype=tf.float32)).numpy()

y_train_pred = scaler_y.inverse_transform(y_train_pred_scaled)
y_test_pred = scaler_y.inverse_transform(y_test_pred_scaled)

y_train_pred = y_train_pred.ravel()
y_test_pred = y_test_pred.ravel()

r2_train_tf = r2_score(y_train, y_train_pred)
```

```

r2_test_tf = r2_score(y_test, y_test_pred)

mae_train_tf = mean_absolute_error(y_train, y_train_pred)
mae_test_tf = mean_absolute_error(y_test, y_test_pred)

mape_train_tf = np.mean(np.abs((y_train - y_train_pred) / y_train)) * 100
mape_test_tf = np.mean(np.abs((y_test - y_test_pred) / y_test)) * 100

print(f"R2 train: {r2_train_tf:.6f}, R2 test: {r2_test_tf:.6f}")
print(f"MAE train: {mae_train_tf:.6f}, MAE test: {mae_test_tf:.6f}")
print(f"MAPE train: {mape_train_tf:.6f}%, MAPE test: {mape_test_tf:.6f}%")

```

Результат работы показан на рис. 2.8.1.

```

R2 train: 0.951393, R2 test: 0.955514
MAE train: 2275.802041, MAE test: 2087.433285
MAPE train: 4.923855%, MAPE test: 4.938934%

```

Рисунок 2.8.1 - Метрики R2, MAE и MAPE для TensorFlow модели.

Коэффициент детерминации R2 показывает, что модель объясняет около 95% вариации зарплаты как на тренировочной выборке (R2 train ~ 0.951), так и на тестовой (R2 test ~ 0.956), что указывает на отсутствие переобучения. Средняя абсолютная ошибка MAE также сопоставима для обучения и теста (MAE train ~ 2276 и MAE test ~ 2087), что подтверждает стабильность предсказаний. MAPE ~ 5% показывает небольшую относительную ошибку и схожие значения для обеих выборок, что дополнительно подтверждает точность модели.

Построим диаграмму рассеяния с предсказаниями TensorFlow модели.

```

X_train_sorted = np.sort(X_train_scaled.squeeze())
y_pred_sorted = model_tf(tf.constant(X_train_sorted, dtype=tf.float32)).numpy()

X_plot_scaled = np.linspace(X_train_scaled.min(), X_train_scaled.max(),
300).reshape(-1, 1)
y_plot_pred_scaled = model_tf(tf.constant(X_plot_scaled,
dtype=tf.float32)).numpy()

X_plot_original = scaler_X.inverse_transform(X_plot_scaled)
y_plot_pred_original = scaler_y.inverse_transform(y_plot_pred_scaled)

plt.scatter(X_train, y_train, alpha=0.6, label='Исходные данные')
plt.plot(X_plot_original, y_plot_pred_original, color='red', linewidth=2,
label='TensorFlow модель')
plt.title('Диаграмма рассеяния с линией TensorFlow модели')
plt.xlabel('Признак X')
plt.ylabel('Целевая переменная y')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Результат работы показан на рис. 2.8.2.

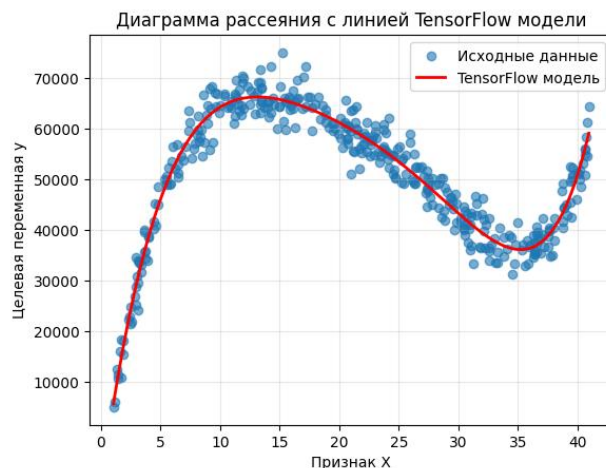


Рисунок 2.8.2 - Диаграмма рассеяния с линией TensorFlow модели.

TensorFlow модель также хорошо описывает данные: кривая проходит через центр облака точек и корректно воспроизводит нелинейный характер зависимости. Точки равномерно распределены относительно кривой без систематических отклонений, разброс выглядит случайным, а плавность кривой подтверждает отсутствие переобучения. Визуальное качество моделирования соответствует высоким значениям метрик ($R^2 \sim 95.1\%$, $MAPE \sim 5\%$), рассчитанных в п. 2.8.

2.9. Сравнение результатов, полученных с/без конструирования полиномиальных признаков.

Обобщим все полученные метрики у полинома 6 степени и TensorFlow модели (см. Таб. 2).

Таблица 2 - Метрики обученных моделей.

Модель	R2 train	R2 test	MAE train	MAE test	MAPE train, %	MAPE test, %
Полином. регрессия	0.951393	0.955516	2275.845058	2087.411498	4.924428	4.939506
TensorFlow модель	0.951393	0.955514	2275.802041	2087.433285	4.923855	4.938934

Построим общую диаграмму рассеяния с предсказаниями обеих моделей.

```
X_sorted_original = scaler_X.inverse_transform(X_train_sorted.reshape(-1, 1))
```

```

y_pred_sorted_original = scaler_y.inverse_transform(y_pred_sorted.reshape(-1,
1))

plt.scatter(X_train, y_train, alpha=0.6, label='Исходные данные', s=30)

plt.plot(X_plot_original, y_plot_pred_original, color='red', linewidth=4,
label='TensorFlow модель')
plt.plot(X_sorted_original, y_pred_sorted_original, color='blue', linewidth=2,
label='Полином 6 степени')

plt.xlabel('Признак X')
plt.ylabel('Целевая переменная y')
plt.title('Сравнение предсказаний моделей на обучающих данных')
plt.legend()
plt.grid(alpha=0.3)
plt.show()

```

Результат работы показан на рис. 2.9.

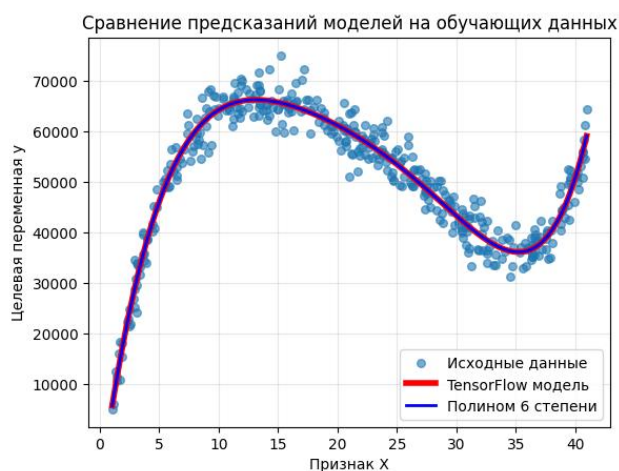


Рисунок 2.9 - Сравнение предсказаний полиномиальной регрессии и модели TensorFlow.

Метрики и график показывают, что обе модели практически одинаковые: значения R^2 совпадают до 4-го знака (~ 0.9514 / ~ 0.9555), MAE различается менее чем на 0.05, а MAPE - менее чем на 0.001%. На графике кривые полиномиальной регрессии и TensorFlow модели полностью накладываются друг на друга, что визуально подтверждает совпадение метрик. Это объясняется тем, что обе модели по сути являются полиномами 6 степени - разница только в способе нахождения коэффициентов: аналитическое решение через конструирование признаков и численная оптимизация градиентным спуском в TensorFlow. Оба метода привели к практически одинаковому результату.

Ссылка на полный код находится в Приложении.

Выводы

В ходе работы были изучены методы линейной и нелинейной регрессии и проведено их применение к двум наборам данных. Линейная регрессия на наборе `lab2_lin1.csv` показала высокую точность предсказания заработной платы на основе стажа, опозданий и KPI. Все рассмотренные методы линейной регрессии показали практически одинаковые результаты: LinearRegression, Lasso и Ridge достигли $R^2 \text{ test} \sim 0.998$, $\text{MAE test} \sim 41.8$ и $\text{MAPE test} \sim 1.09\%$, что подтверждает линейность зависимости и отсутствие необходимости в регуляризации. Модель ElasticNet показала незначительно худшие значения метрик ($R^2 \text{ test} \sim 0.998125$, $\text{MAPE test} \sim 1.095\%$), что связано с более сильным сжатием коэффициентов за счет двойной регуляризации.

Нелинейная регрессия на наборе `lab2_poly2.csv` выявила явный нелинейный характер зависимости между опытом работы и зарплатой. Полиномиальная регрессия шестой степени показала высокое качество: $R^2 \text{ test} \sim 0.956$, $\text{MAE test} \sim 2087$, $\text{MAPE test} \sim 4.94\%$, при этом кривая модели плавно повторяет тренд данных без признаков переобучения. TensorFlow модель, обученная градиентным спуском, показала практически идентичные результаты: $R^2 \text{ test} \sim 0.9555$, $\text{MAE test} \sim 2087$, $\text{MAPE test} \sim 4.94\%$, что подтверждается полным совпадением кривых на графике сравнения.

Сравнение моделей показало, что полиномиальная регрессия и TensorFlow модель дают одинаковое качество предсказаний, поскольку обе по сути являются полиномами 6 степени - разница лишь в способе нахождения коэффициентов: аналитическое решение через конструирование признаков и численная оптимизация градиентным спуском. Обе модели демонстрируют высокую точность и способность к обобщению с $R^2 \sim 95.5\%$ и $\text{MAPE} \sim 5\%$.

ПРИЛОЖЕНИЕ

1. Ссылка на код:

https://colab.research.google.com/drive/1EG68u4gyMYgOZ9I5Ga_EEyQDwza2GB2K?usp=sharing.